

Mendix Development Standards

Internal reference document | Engineering & Architecture | Version 2.1 | Last reviewed: March 2026

This document captures the development standards and conventions that apply across all Mendix applications in our organisation. It is intended as a reference for developers, architects, and team leads. Not every rule applies to every project — use judgment and flag deviations to the architecture team when they arise. This document is updated roughly twice a year; check the version number before relying on specific guidance.

Security

Security mistakes are the hardest to fix after the fact. The rules below apply to all applications. When in doubt, contact the security team before building.

Passwords and authentication

- Always use BCrypt or SSHA256 for password hashing. MD5 and SHA1 are not acceptable.
- Minimum password length is 12 characters. Require at least one uppercase letter, one number, and one special character.
- For applications with an internal user base, use the company SSO where technically feasible. Avoid building custom login flows.
- Set Project Security to Production mode before deploying to acceptance or production. Prototype mode is only for local development.

Access control

- Multi-tenant applications must use `CurrentUser` in entity access rules. Never use `[%CurrentUser%]` as a workaround without understanding the security implications.
- Anonymous users must not be able to create records that persist in the database.
- Anonymous users must not be able to edit unlimited string attributes — this is a common injection vector.
- Entity access rules must be applied in document generation flows (quotes, invoices, contracts). This is easy to miss and hard to audit retrospectively.
- Default access for new entity members should be `None` or `Read`, not `Write`. Change explicitly where write access is needed.

Credentials

- Never store API keys, passwords, tokens, or connection strings in source code, configuration files, or application logs. Use the secrets management tooling provided by the platform team.

Session management

- Sensitive applications (payments, personal data, HR) should use a session timeout of around 15 minutes.
- Standard internal tools can use 60 minutes. Public-facing apps should be assessed individually.

Architecture and integrations

These conventions are based on issues we have seen repeatedly in production. The hard rules here exist because breaking them creates problems that are painful to diagnose and expensive to fix.

- Never commit inside a loop. Retrieve all records you need before the loop starts, then commit once after.
- Do not use recursive microflows unless you have a tested and explicit termination condition. Infinite recursion brings down the application.
- External integrations must handle failures gracefully. A failed call to an external API should never corrupt local data silently.
- Log integration calls — at minimum the endpoint, HTTP status, and timestamp. This is essential for incident response.
- Do not call external services synchronously inside loops. Where you need to process a list of records against an external API, batch the calls or use a queue.
- REST APIs we expose to other teams must be versioned from day one. Use /api/v1/ as the path prefix. Backwards-incompatible changes require a new version.
- Avoid tight coupling between modules. A module should be able to be replaced or removed without cascading changes across the application.

Note on performance

Slow operations (anything that may take more than 2-3 seconds) should use asynchronous microflows. This keeps the UI responsive and prevents timeout errors in the browser. Reference data that is frequently read and rarely updated (product catalogues, country lists, pricing tables) should be cached.

Data and domain modeling

- Each entity should belong to a single module. Cross-module entity references create maintenance problems — use association patterns or published services to cross boundaries.
- Store all timestamps in UTC. Timezone conversion should happen in the UI layer, not in the domain model or microflows.
- Boolean attributes must have unambiguous names. Use IsActive, HasConsent, IsVerified. Do not use Status, Flag, or Type as a boolean name.
- Enumerations should be named with the entity they belong to as a prefix: Order_Status, not just Status. This avoids conflicts as the model grows.
- Always add a description to entities and attributes in the domain model. Teams joining the project later — and Maia — rely on this to understand intent.
- Personal data should be stored in dedicated attributes or entities so retention and erasure policies can be applied cleanly.

Domain terminology

To ensure consistency between teams and in AI-generated output, we use the following terms. When Maia generates user stories or acceptance criteria, it should use these terms.

Term	Meaning	Not
Customer	An external person or organisation with at least one confirmed order.	Client, User, Account
Order	A confirmed purchase. States: Draft, Confirmed, Fulfilled, Cancelled.	Purchase, Transaction

Product	A sellable item with a SKU, price, and inventory record.	Item, Article, SKU
Platform admin	An internal user with company-wide Control Center access.	Admin, Super user

Development conventions

Naming

- Microflows: [Module]_[Entity]_[Action] — for example Order_Invoice_Create or Customer_Profile_Update.
- Pages: [Module] - [Entity] - [View type] — for example Order - Invoice - Overview or Customer - Profile - Edit.
- Modules should have a name that reflects their domain responsibility. Technical infrastructure modules should be clearly distinguished from domain modules.

Code quality

- Each module should have one clear responsibility. Do not mix domain logic with technical plumbing in the same module.
- Keep microflow depth to around 3 levels. Beyond that, extract named sub-flows.
- Every microflow that changes domain state should have at least one unit test covering the main path and one covering the most likely error case.
- Run the Mendix Best Practice Recommender before submitting work for review. Findings do not need to be resolved but should be documented if intentionally ignored.

Marketplace modules

- Only use Marketplace modules that are marked as Mendix-supported or that have been approved by the architecture team. Check the approval list before adding a new module.
- Be aware that some modules carry additional runtime or infrastructure costs (event brokers, for example). Flag this to the project lead before adopting them.

Compliance and data privacy

We operate under GDPR and are subject to periodic audits. The points below are the most commonly applicable requirements. For applications that process sensitive personal data, involve the compliance team before building.

- Collect only the personal data that is actually needed for the service. Do not collect data speculatively.
- Implement the right to erasure in all applications that store personal data. This must be designed in — it cannot be retrofitted cleanly.
- Data retention policies must be automated. Relying on manual cleanup is not acceptable.
- Log all access to personal data for audit purposes. This includes read access, not just writes.
- Maintain an audit trail for create, update, and delete operations on business-critical entities.
- Encrypt personal data at rest and in transit.

Design and UI standards

- Use the company UI theme and design tokens. Do not override theme variables directly in pages or snippets — raise a request with the design system team if you need a variant.
- Do not modify the standard Marketplace design module. Create a custom UI module for company-specific overrides.
- Use Atlas UI building blocks as the starting point for new pages. Custom layouts should be a deliberate choice, not the default.
- Customer-facing pages must meet WCAG 2.1 AA accessibility requirements: adequate colour contrast, keyboard navigability, and labels for screen readers.
- Page naming follows the convention: [Module] - [Entity] - [Action]. For example: Order - Invoice - Overview.